# RL basics: a quick-reference glossary

Short definitions for the Wall-E RL project's CartPole phase. Read across the groups in order. A CartPole example follows terms where it helps. These are general learning terms; CartPole results are not physical-robot validation.

## The control loop

**Agent** — The decision-maker being trained. Here, the PPO-controlled CartPole policy is the agent.

**Environment** — The world the agent interacts with. Here, Gymnasium's CartPole computes how the cart and pole move after a push.

**State** — A complete description of the world sufficient to predict what can happen next, given an action. It can include information the agent cannot see.

**Observation** — The information supplied to the agent at one step. CartPole supplies cart position, cart velocity, pole angle, and pole angular velocity; an observation is not always a complete state.

**Action** — The command chosen by the agent. In CartPole, 0 pushes left and 1 pushes right.

**Action space** — The set of allowed actions. CartPole's is Discrete(2); a future motor command may be continuous.

**Transition** — One move from current state to next state after an action, together with its reward. This is one environment interaction.

**Reward** — A number returned for one transition that defines the task's immediate feedback. CartPole gives +1 for each step, including the step that ends in failure in our default setup.

**Timestep (environment step)** — One action followed by one environment update. It is not an episode or a neural-network optimizer step.

**Episode** — One attempt, from reset() until the task terminates or an outside limit truncates it.

**Trajectory / rollout** — A recorded sequence of observations, actions, and rewards. PPO gathers rollout blocks that can cross episode boundaries.

**Termination** — An episode ends because of a task condition, such as CartPole's pole falling too far.

**Truncation** — An episode is cut off by a limit outside the task's failure condition, such as CartPole-v1's 500-step time limit.

## Goals and decisions

**Return** — The sum of rewards from a point onward. An episode score is often the undiscounted return; learning may use discounted return.

**Discount factor (gamma)** — The number gamma that weights a reward one step later by gamma, two steps later by $\text{gamma}^2$, and so on. Our PPO config uses 0.99.

**Policy** — A rule mapping an observation to an action or an action distribution. In our project, it is represented by a neural network.

**Stochastic policy** — A policy that outputs action probabilities and samples an action. If left has probability 0.3, the same observation can sometimes produce left and sometimes right.

**Deterministic policy / action selection** — For a fixed observation and policy weights, chooses the same action. Our CartPole evaluation selects the actor's most probable action.

**Exploration** — Trying actions to learn their consequences. PPO training samples from its stochastic policy; evaluation ordinarily measures preferred actions.

**Exploitation** — Choosing an action believed to work well using what the policy has learned so far. Exploration and exploitation describe behavior, not separate network files.

**Seed** — The starting number for a pseudorandom generator. It helps repeat initial states, sampled actions, and training runs under compatible conditions; it is not the number of steps.

**Expected return** — The average return one would get over the randomness in starts, actions, and outcomes when following a policy.

## Values and learning signals

**Value function, `V(s)`** — The expected discounted return from a state when the specified policy continues. The same state can have different values under different policies.

**Action-value function, `Q(s,a)`** — The expected discounted return when action a is taken first in state s, then the specified policy continues.

**Bellman equation** — A consistency relationship: value now equals expected immediate reward plus discounted value after the transition.

**Bellman target** — A reward-plus-next-value estimate used to train a value predictor, with no next-value contribution after a true terminal transition.

**Critic** — The network that estimates future return, usually V(s) in our PPO setup. It helps training; it does not choose CartPole's push.

**Advantage, `A(s,a)`** — How much better action a is than the policy's average action from s: conceptually Q(s,a) - V(s). Training uses an estimate.

**Temporal-difference (TD) error** — The difference between a one-step target and the current value prediction: reward + gamma × next value − current value, adjusted at episode boundaries.

**Bootstrapping** — Using a current estimate of the next state's value to help estimate the present value, instead of waiting for every future reward.

**Generalized Advantage Estimation (GAE)** — PPO's way of combining TD errors across several steps to estimate advantage. gae_lambda controls how strongly later errors contribute.

## Policy training and PPO

**Actor** — The network that converts an observation into an action distribution. CartPole's actor gives probabilities for left and right.

**Network weights** — Adjustable numbers inside the actor and critic. Training changes them; inference and evaluation leave them fixed.

**Training** — Collecting experience and updating weights to improve a learning objective. training/train.py creates a fresh model on each invocation.

**Inference** — Running a trained actor to select an action without updating weights. training/demo.py does this.

**Evaluation** — Measuring a policy on episodes without changing weights. Separate seeds and environments test how well it works beyond training experience.

**PPO (Proximal Policy Optimization)** — An actor-critic training algorithm that collects experience with its current policy and uses a clipped objective to discourage overly large probability changes during updates.

**On-policy** — Training mainly on experience gathered by the current version of the policy. PPO collects fresh rollout data after updates rather than indefinitely reusing old rollouts.

**Rollout buffer** — Temporary storage for the current block of transitions used by PPO's update. It is not a long-lived archive of all past episodes.

**Loss / objective** — A numerical quantity the optimizer changes weights to improve. PPO combines actor, critic, and optional entropy terms; a low loss alone is not proof of good control.

**Optimizer** — The procedure that adjusts network weights using gradients. Our installed PPO uses Adam.

**Learning rate** — A setting controlling the scale of optimizer updates; it is not the amount every individual weight moves.

**Minibatch** — A small group of collected transitions processed in one optimizer step. Our config uses 64 samples.

**Epoch** — One pass through the collected rollout data during optimization. Our config makes 10 passes per rollout.

**Clipping (PPO)** — A limit in the actor's training objective on the benefit of changing an action's probability ratio too far. It is not a hard cap on weights, probabilities, or motor output.

**Entropy** — A measure of how spread out an action distribution is. [0.5, 0.5] has more entropy than [0.99, 0.01].

**Checkpoint** — A saved snapshot of model weights and training state at a point in the run. A periodic checkpoint need not be the best policy.

**Validation** — Evaluation performed during training on a separate environment. Our callback uses its mean reward to save a best_model.zip.

**Holdout evaluation** — Evaluation on fresh episodes and seeds reserved from training and routine validation. It measures generalization within CartPole, not physical transfer.

**Generalization** — Working on starts or conditions beyond those used to fit the policy. A high score on one seed does not establish broad reliability.

## From simulation toward Wall-E

**Simulator / physics model** — Software that predicts how actions change a virtual system. CartPole is the current example; the future robot model requires measured physical parameters.

**Deployable observation** — An input that the physical robot can actually measure or estimate reliably. Our TT motors have no encoders, so wheel position and speed cannot simply be assumed available.

**Sim-to-real gap** — Differences between simulated and physical dynamics, sensors, timing, and actuation that can make a successful simulated policy fail on hardware.

**Safety cutoff** — An independent rule that stops or limits unsafe robot behavior. It must remain outside the learned policy.

## Sources and next reading

- OpenAI Spinning Up: Key Concepts in RL — agent, environment, policy, return, values, Bellman equations, and advantage.
- Gymnasium basic usage — reset(), step(), observations, rewards, termination, and truncation.
- Stable-Baselines3 PPO — PPO settings and implementation terminology.
- Our CartPole walkthrough, workflow PDF, and Bellman PDF — longer explanations and runnable examples.